Package 'golemstockindices'

November 7, 2026

Title Interactive Explorer of the Major Stock Market Indices

Version 0.1.0

Maintainer Dimitrios Zacharatos <dzacharatos@yahoo.com>

Description A 'shiny' application for exploring the major stock market indices of the US, Europe and Asia, such as the S&P 500, the Euro Stoxx 50 and the Nikkei 225, and the VIX. Shows where every index stands against its last close, the start of the year, a year ago and its all-time high, and plots interactive candlestick and volume charts of the whole history of each. Also compares normalized indices, and shows a calendar of monthly returns, their seasonality by day of the week, week of the year, month and year of the US presidential cycle, whether an index's recent volatility is normal, with a GARCH forecast, and its falls from its all-time high. Daily prices are retrieved from 'Yahoo Finance'.

Imports bslib,
cachem,
curl,
DT,
htmlwidgets,
jsonlite,
plotly,
shiny,
shinycssloaders,
stats,
tools,
utils

Suggests htmltools,
testthat (>= 3.0.0)

Config/testthat/edition 3

License GPL-3

URL https://sedzinfo.github.io/golemstockindices/, https://github.com/sedzinfo/golemstockindices

BugReports https://github.com/sedzinfo/golemstockindices/issues

Encoding UTF-8

Roxygen list(markdown = TRUE)

Config/roxygen2/version 8.1.1

Contents

drawdown . . . 2
drawdown_analysis . . . 3
drawdown_episodes . . . 4
drawdown_table . . . 5
drawdown_text . . . 5
get_history . . . 6
get_indices . . . 7
index_summary . . . 7
monthly_returns . . . 8
moving_average . . . 9
normalize_prices . . . 9
overview_table . . . 10
plot_compare . . . 11
plot_drawdowns . . . 11
plot_returns_calendar . . . 12
plot_seasonality . . . 13
plot_timeseries . . . 13
plot_volatility . . . 14
run_app . . . 15
seasonality_analysis . . . 16
seasonality_table . . . 17
seasonality_text . . . 18
seasonal_returns . . . 19
volatility_analysis . . . 20
volatility_text . . . 22

Index **23**

`drawdown` *Drawdown of prices*

Description

How far each close is below the highest close up to that day, the all-time high so far, in percent: 0 on a new high and -50 at half of it.

Usage

```
drawdown(close)
```

Arguments

`close`	Numeric vector of close prices, sorted by date.

Value

A numeric vector as long as `close`, from -100 to 0.

Examples

```
drawdown(c(100, 120, 90, 130, 65))
```

`drawdown_analysis` *Drawdown analysis of an index*

Description

How far is the index below its all-time high, and how does that compare with its earlier falls? See `drawdown_episodes()` for how the falls are found.

Now: the last close against the all-time high, the lowest close since that high, the rise the price needs to get back to it, and the earlier falls that went at least as deep as the current one has so far: how many days they took to get back to their high.

The history: the falls of at least `min_depth` percent, the deepest first, the share of days that closed that far below the high, and the days that closed at a new all-time high.

Usage

```
drawdown_analysis(history, min_depth = 20)
```

Arguments

`history`	Price history of one index from `get_history()`, with all of it: the highs are those of the history given.
`min_depth`	Smallest fall of the history to list, in percent below the high.

Value

`NULL` when there are fewer than 2 prices, or a list with:

index, first_date, last_date, min_depth What was analysed.

now `date` and `close` of the last day, `drawdown_pct`, `high_date` and `high_price` of the all-time high, `days_since_high`, and `rise_needed_pct` to get back to it. While the price is below the high also `low_date` and `low_price`, the lowest close since the high, `deepest_pct` how far that is below it, `rank` of the current fall among all the falls of the history by that depth, 1 for the deepest, and `earlier`, the earlier falls at least as deep, as rows of `drawdown_episodes()`. At the high, `previous` holds the last fall of at least `min_depth` percent, or no rows.

episodes The falls of at least `min_depth` percent from `drawdown_episodes()`, with their `rank` in front, 1 for the deepest.

series A row per day: `date`, `close`, `high` (the all-time high so far) and `drawdown_pct` (see `drawdown()`).

share_under Percent of the days that closed at least `min_depth` percent below the high.

new_highs Days that closed at a new all-time high, the first day of the history left out.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
analysis <- drawdown_analysis(spx)
analysis$now$drawdown_pct
analysis$episodes
drawdown_text(analysis)
```

`drawdown_episodes` *Falls of the price from its all-time high*

Description

Splits a price history into its falls, the drawdown episodes. A fall starts on the last close at an all-time high, the peak, goes down to its lowest close, the trough, and ends on the first close back at or above the peak. The last fall is still going on when the price has not got back to its high. The highs are those of the history given: for an index whose history starts after its real all-time high, the first fall starts on the first day.

Usage

```
drawdown_episodes(close, date, min_depth = 0)
```

Arguments

`close`	Numeric vector of close prices, sorted by date.
`date`	Dates of the closes.
`min_depth`	Smallest fall to keep, in percent below the peak: 20 keeps the falls that went at least 20% down. The default, 0, keeps them all.

Value

A data frame with a row per fall, the deepest first: `peak_date` and `peak_price`, `trough_date` and `trough_price`, `depth_pct` (from -100 to 0), `days_to_trough` (from the peak), `recovery_date` (`NA` while the fall goes on), `days_to_recover` (from the trough), `days_total` (from the peak to the recovery, or to the last date while the fall goes on) and `ongoing`.

Examples

```
drawdown_episodes(c(100, 120, 90, 130, 65, 70), as.Date("2026-01-01") + 0:5)
# only the falls of at least 40%
drawdown_episodes(c(100, 120, 90, 130, 65, 70), as.Date("2026-01-01") + 0:5, min_depth = 40)
```

drawdown_table *Table of the falls of an index*

Description

A sortable table of the falls of `drawdown_analysis()`, the deepest first, with the numbers of the chart of `plot_drawdowns()`. The fall is shaded red, more strongly the deeper it is, and the current fall, which has not got back to its high, is in bold.

Usage

```
drawdown_table(analysis)
```

Arguments

`analysis`	The result of `drawdown_analysis()`.

Value

A DT datatable.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
drawdown_table(drawdown_analysis(spx))
```

drawdown_text *The drawdown analysis in words*

Description

Sentences built from the numbers of `drawdown_analysis()`, for the Drawdowns tab.

Usage

```
drawdown_text(analysis)
```

Arguments

`analysis`	The result of `drawdown_analysis()`.

Value

A list with `now` and `history`, each with a `headline` and `points`, a character vector of sentences.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
drawdown_text(drawdown_analysis(spx))
```

`get_history` *Get the daily price history of indices*

Description

Returns the daily prices of one or more stock market indices from Yahoo Finance, in index points. Each index's history is kept on disk between R sessions, and once it is more than an hour old only its last two weeks are downloaded again, so calls are fast. An index is priced on the days its exchange trades, so weekends and holidays have no prices.

Usage

```
get_history(labels, start_date = Sys.Date() - 365, indices = get_indices())
```

Arguments

`labels`	Character vector of index labels, such as "S&P 500", from the `label` column of `get_indices()`.
`start_date`	Date. The first day to get, a year ago by default.
`indices`	Data frame of indices from `get_indices()`, to look the labels up.

Value

A data frame with one row per index and day, sorted by index and date, with the `open`, `high`, `low`, `close` and `volume` columns, `timestamp` as a Date and the `label`. Labels that match no index are left out with a message, and `NULL` is returned when no label matches or nothing could be downloaded.

Examples

```
## Not run:
spx <- get_history("S&P 500", as.Date("2020-01-01"))
plot_timeseries(spx)

history <- get_history(c("S&P 500", "DAX", "Nikkei 225"))
plot_compare(normalize_prices(history))

## End(Not run)
```

get_indices *The stock market indices of the app*

Description

The major indices of the US, Europe and Asia, and the VIX, with their symbols on Yahoo Finance. The `label` column holds the names, such as "S&P 500", that pick indices in `get_history()`.

Usage

```
get_indices()
```

Value

A data frame with a row per index: `label`, `symbol` on Yahoo Finance, `region` ("US", "Europe" or "Asia") and the `currency` of the shares in it. The VIX is a measure of the expected volatility of the S&P 500, in percent a year, so its currency is `NA`.

Examples

```
get_indices()
```

index_summary *Where each index stands*

Description

For each index of a price history: its last close, and the change in percent since the close before, since the last close of the year before (year to date) and since the last close at least a year before, and how far it is below its highest close. Pass the whole history, as the highest close is that of the history given.

Usage

```
index_summary(history)
```

Arguments

`history`	Price history from `get_history()`, with `label`, `timestamp` and `close` columns.

Value

A data frame with a row per index, in the order of their first rows in `history`: `label`, `date` and `close` of the last day, `day_pct`, `ytd_pct`, `year_pct`, `high` and `high_date` of the highest close, `from_high_pct` (from -100 to 0) and `first_date`, the first day of the history. A change is `NA` when the history does not go back far enough.

Examples

```
history <- data.frame(
  label = "Alpha",
 timestamp = as.Date(c("2024-12-31", "2025-06-30", "2025-12-31", "2026-01-02", "2026-01-05")),
  close = c(100, 150, 120, 126, 132)
)
index_summary(history)
```

`monthly_returns` *Monthly returns of an index*

Description

Each month's return is its last close relative to the last close of the month before, in percent. The first month, and the months still to come, are `NA`.

Usage

```
monthly_returns(history)
```

Arguments

`history`	Price history of one index from `get_history()`, with `timestamp` and `close` columns, sorted by date.

Value

A year by month matrix of returns in percent, rounded to 2 decimals, with the years as row names and `month.abb` as column names.

Examples

```
history <- data.frame(
  timestamp = as.Date(c("2024-01-31", "2024-02-29", "2024-03-31")),
  close = c(100, 110, 121)
)
monthly_returns(history)
```

moving_average *Moving average of prices*

Description

The simple moving average: each day's value is the mean of the last `days` closes, that day included, so the first `days - 1` days have none.

Usage

```
moving_average(close, days)
```

Arguments

`close`	Numeric vector of close prices, sorted by date.
`days`	Number of days to average, such as 50 or 200.

Value

A numeric vector as long as `close`, `NA` for the first `days - 1` days, and all `NA` when there are fewer than `days` closes, as for an index younger than the average.

Examples

```
moving_average(c(1, 2, 3, 4, 5), days = 3)
```

normalize_prices *Normalize close prices to 100*

Description

Every index starts at 100 on the first date on which all of them have data, so their lines can be compared directly: 150 means the price rose by half since that date.

Usage

```
normalize_prices(history)
```

Arguments

`history`	Price history from `get_history()`, with `label`, `timestamp` and `close` columns, sorted by index and date.

Value

`history` from the first common date on, with an `index` column.

Examples

```
history <- data.frame(
  label = rep(c("Alpha (A)", "Beta (B)"), each = 3),
  timestamp = rep(as.Date("2024-01-01") + 0:2, 2),
  close = c(10, 11, 12, 200, 180, 220)
)
normalize_prices(history)
```

`overview_table` *Table of where the indices stand*

Description

A table with a row per index of `index_summary()`, in the order of `get_indices()`: its region, last close and its date, the change on the day, this year and over a year, how far it is below its all-time high, that high and its date, the currency of its shares and the first day of its history. The changes are shaded red for falls and green for rises, each column on its own scale, as a day's moves are much smaller than a year's, and the distance from the high red, more strongly the further it is.

Usage

```
overview_table(summary, indices = get_indices())
```

Arguments

`summary`	The result of `index_summary()`.
`indices`	Data frame of indices from `get_indices()`, for the regions and currencies.

Value

A DT datatable.

Examples

```
indices <- get_indices()
history <- get_history(indices$label[1:3], as.Date("1900-01-01"))
overview_table(index_summary(history), indices)
```

plot_compare *Line chart of normalized close prices*

Description

One line per index, with buttons that switch the y axis between a linear and a log scale.

Usage

```
plot_compare(prices, mode = "light")
```

Arguments

prices	Prices from normalize_prices().
mode	"light" (the default) or "dark", to suit a light or dark page.

Value

A plotly object.

Examples

```
history <- get_history(c("S&P 500", "Euro Stoxx 50", "Nikkei 225"))
plot_compare(normalize_prices(history))
```

plot_drawdowns *Drawdown chart of an index*

Description

Two panels over the whole history. The upper one is the close on a log scale with the all-time high so far; the lower one is the drawdown, how far each close is below that high (see drawdown()), with a dashed line at min_depth percent. The falls of the analysis are shaded from their peak to their recovery, the current one in amber, and their troughs carry their rank, as in drawdown_table(); hover over a trough for its dates.

Usage

```
plot_drawdowns(analysis, mode = "light")
```

Arguments

analysis	The result of drawdown_analysis().
mode	"light" (the default) or "dark", to suit a light or dark page.

Value

A plotly object.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
plot_drawdowns(drawdown_analysis(spx))
```

`plot_returns_calendar` *Heatmap of monthly returns by year*

Description

Months are colored from red to green by their return. The color limit is the 95th percentile of the absolute returns, so a few extreme months do not wash out the rest.

Usage

```
plot_returns_calendar(calendar, label = NULL, mode = "light")
```

Arguments

`calendar`	Matrix from `monthly_returns()`.
`label`	Label of the index for the title, or `NULL` for none.
`mode`	`"light"` (the default) or `"dark"`, to suit a light or dark page.

Value

A plotly object.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
plot_returns_calendar(monthly_returns(spx), "S&P 500")
```

plot_seasonality *Seasonality chart of an index*

Description

Two panels with a bar per group of `seasonality_analysis()`. The upper one shows the mean return, green above 0 and red below, with the median as a diamond; the lower one shows the share of the returns above 0, with a dashed line at 50%. Hover over a bar for all the numbers of its group.

Usage

```
plot_seasonality(analysis, mode = "light")
```

Arguments

`analysis`	The result of `seasonality_analysis()`.
`mode`	`"light"` (the default) or `"dark"`, to suit a light or dark page.

Value

A plotly object.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
plot_seasonality(seasonality_analysis(spx, "month"))
plot_seasonality(seasonality_analysis(spx, "day"), mode = "dark")
```

plot_timeseries *Candlestick chart of an index's prices*

Description

Plots the daily open, high, low and close, in index points, with a line through the close, its moving averages, date range buttons and a range slider. Underneath, smaller charts show the trading volume, when the history has any, and the drawdown: how far the index is below its all-time high so far. A click on a legend entry hides or shows that line.

The vertical axes follow the days in view: zoomed in, each runs from the lowest to the highest value of those days, so a week's candles fill the chart instead of lying flat on an axis that reaches back to the start of the history. The Linear and Log buttons switch the axis of the prices; on the log axis equal distances are equal percentages, which suits the whole history.

Usage

```
plot_timeseries(
  history,
  mode = "light",
  averages = c(50, 200),
  show_drawdown = TRUE
)
```

Arguments

history	Price history of one index from get_history(). The averages and the drawdown use the whole history, so pass all of it and zoom with the range buttons.
mode	"light" (the default) or "dark", to suit a light or dark page.
averages	Days of the moving averages to draw (see moving_average()), or NULL for none.
show_drawdown	TRUE to add the drawdown chart (see drawdown()).

Value

A plotly object.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
plot_timeseries(spx)
# shorter averages, and no drawdown chart
plot_timeseries(spx, averages = c(20, 100), show_drawdown = FALSE)
```

---

plot_volatility *Volatility chart of an index*

Description

Three panels. The upper one is the volatility of every month_days-day period over the whole history, annualized, with its median and 95th percentile: where the line is now shows how rough the last month was. The middle one shows each of the last days days as a bar of its move in the standard deviations the GARCH model expected for it, red for falls and green for rises, with lines at 2 and 4 standard deviations. The lower one shows the last context_days closes and the forecast of the next days: the median close, and the ranges of the middle half and of probability level (see volatility_analysis()). Without a GARCH model only the upper panel is drawn.

Usage

```
plot_volatility(analysis, mode = "light", days = 63, context_days = 21)
```

Arguments

analysis	The result of volatility_analysis().
mode	"light" (the default) or "dark", to suit a light or dark page.
days	Days of the middle panel.
context_days	Closes before the forecast in the lower panel.

Value

A plotly object.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
plot_volatility(volatility_analysis(spx))
```

---

run_app *Launch the Stock Market Indices app*

Description

Starts an interactive 'shiny' application for exploring the major stock market indices of the US, Europe and Asia, with daily prices from Yahoo Finance.

Usage

```
run_app(analytics = FALSE, keep_alive = TRUE, ...)
```

Arguments

analytics	TRUE to add Google Analytics, through the cookie banner shared with the sites of sedzinfo.github.io, which loads it only after the visitor accepts. Meant for the public deployment (see app.R); off by default, so that running the app yourself sends nothing.
keep_alive	TRUE (the default) to ping the server every 15 seconds while the page is open, so that proxies do not close idle connections. FALSE lets the app go to sleep when nobody uses it, as app.R does.
...	Further arguments passed to shiny::shinyApp(), for example options = list(port = 8080).

Details

The application has the following tabs:

Overview A table of all the indices: the last close, the change on the day, this year and over a year, and how far each is below its all-time high.

Timeseries Candlestick chart of the daily open, high, low and close of an index over its whole history, with its 50- and 200-day moving averages, and charts of the volume and of the drawdown (how far the index is below its all-time high) underneath.

Compare Closes of up to 10 indices, normalized to 100 on the first date all of them have data.

Returns calendar Monthly returns by year for the index selected in the Timeseries tab.

Seasonality The returns of the index selected in the Timeseries tab by day of the week, week of the year, month or year of the US presidential cycle, with a test of whether the differences could be chance.

Volatility Whether the volatility of the last weeks and the move of the last days of the index are normal, and a GARCH forecast of the next days with the value at risk.

Drawdowns How far the index is below its all-time high, and how deep its earlier falls went and how long they took to recover.

The prices are downloaded from Yahoo Finance when an index is first picked and kept on disk between R sessions, so after the first download only the last two weeks are fetched, at most once an hour. No API key is needed, but an internet connection is.

Value

A `shiny::shinyApp()` object. Printing it, or calling `run_app()` at the console, opens the app in the browser.

Examples

```
if (interactive()) run_app()

# on a fixed port
if (interactive()) run_app(options = list(port = 8080))
```

---

seasonality_analysis *Seasonality analysis of an index*

Description

Do some days of the week, weeks of the year, months or years of the US presidential cycle bring better returns than others? The returns of the complete periods (see `seasonal_returns()`) are summed up by group, and a Kruskal-Wallis test asks whether the differences between the groups are larger than chance would give. The test compares the ranks of the returns, so a few extreme months do not decide it, but it takes the returns as independent, which market returns are only roughly: calm and wild stretches come in runs.

Usage

```
seasonality_analysis(history, period = c("day", "week", "month", "year"))
```

Arguments

history	Price history of one index from get_history(), with all of it.
period	"day", "week", "month" or "year".

Value

NULL when the history has no complete period with one before it, or a list with:

index, period What was analysed.

returns The returns of seasonal_returns().

groups A row per group that has returns, in their natural order: group, returns (how many), mean_pct, median_pct, up_pct (the share of the returns above 0), best_pct and worst_pct.

all The same numbers for all the returns together, as a list.

p_value The p-value of the Kruskal-Wallis test, NA when fewer than two groups have returns, or no group has two.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
analysis <- seasonality_analysis(spx, "month")
analysis$groups
analysis$p_value
seasonality_text(analysis)
```

---

seasonality_table *Table of the seasonality of an index*

Description

A table with a row per group of seasonality_analysis(), in their natural order, with the numbers of the chart of plot_seasonality(). The mean and the median are shaded red below 0 and green above it, and the share up red below 50% and green above it, more strongly the further they are from the middle.

Usage

```
seasonality_table(analysis)
```

Arguments

analysis	The result of seasonality_analysis().

Value

A DT datatable.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
seasonality_table(seasonality_analysis(spx, "month"))
```

`seasonality_text` *The seasonality analysis in words*

Description

Sentences built from the numbers of `seasonality_analysis()`, for the Seasonality tab: the best and the worst group, and whether the differences between the groups could be chance.

Usage

```
seasonality_text(analysis)
```

Arguments

`analysis`	The result of `seasonality_analysis()`.

Value

A list with `groups` and `chance`, each with a `headline` and `points`, a character vector of sentences.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
seasonality_text(seasonality_analysis(spx, "day"))
```

`seasonal_returns` *The return of each complete period of an index*

Description

A period's return is its last close relative to the last close of the period before, in percent, as in `monthly_returns()`. Markets close at weekends and on holidays, so a day's return runs from the close of the trading day before, as long as it is at most 4 calendar days earlier: a weekend or a long weekend, but not a gap in the data. Only complete periods count: the period must have ended, either because the history goes on after it or because its last close is on its last weekday, and the period before must be the one just before it. So the first period, which has none before it, and the current one, which has not ended, are left out.

Each return is put in a group: the day of the week for days, the ISO 8601 week of the year for weeks (Monday to Sunday), the month for months, and the year of the 4-year US presidential cycle for years. The elections come in years divisible by 4, such as 2024, so 2025 was a post-election year, 2026 a midterm year and 2027 a pre-election year.

Usage

```
seasonal_returns(history, period = c("day", "week", "month", "year"))
```

Arguments

`history`	Price history of one index from `get_history()`, with `timestamp` and `close` columns.
`period`	`"day"`, `"week"`, `"month"` or `"year"`.

Value

A data frame with a row per complete period, the oldest first: `start` and `end`, the first and last day of the period, `group`, a factor with the groups in their natural order, and `return_pct`.

Examples

```
# the weekdays of 2024 to the end of April
days <- as.Date("2024-01-01") + 0:120
days <- days[as.POSIXlt(days)$wday %in% 1:5]
history <- data.frame(timestamp = days, close = 100 * 1.001^seq_along(days))
seasonal_returns(history, "month")
head(seasonal_returns(history, "day"))
```

`volatility_analysis` *Volatility analysis of an index*

Description

Is the index's recent volatility normal? The analysis looks at the daily log returns, `log(close(t) / close(t-1))`, from the close of one trading day to the next, in two ways. All the windows count trading days, and a year is 252 of them.

The last month (`month_days` trading days, 21 by default): its volatility, the standard deviation of the daily returns times `sqrt(252)`, against the year before it and against every past period of the same length; an F test and the Fligner-Killeen test, which is robust to a few extreme days, of whether its variance is larger than the year before; its largest daily move, in standard deviations of the year before; and its return against every past period of the same length.

The last days (`recent_days` days): their return against every past period of as many days; the same, in standard deviations of the year before; a GARCH(1,1) model with Student-t errors (see `garch_fit()`), fitted to the `garch_years` years before them, which judges each day against the volatility the model expected for it given the days before; and each day's volume against the `volume_days` days before it (see `volume_ratio()`), judged by how often the index's volume was as far from usual.

The next days (`forecast_days` days): the same model, with its variance brought up to the last day, forecasts the volatility and the range of the price (see `garch_forecast()`), and gives the value at risk and the expected shortfall of the next day (see `garch_risk()`). A Kupiec test (see `kupiec_test()`) checks how often the model's value at risk was exceeded over the last year. The value at risk and expected shortfall of the last year's daily returns are given without the model too.

Usage

```
volatility_analysis(
  history,
  month_days = 21,
  recent_days = 2,
  garch_years = 4,
  forecast_days = 5,
  level = 0.95,
  volume_days = 21
)
```

Arguments

`history`	Price history of one index from `get_history()`, sorted by date. The comparisons use the whole history, so pass all of it.
`month_days`	Trading days of the volatility window, the "month": 21, or for example 5 for the last week or 63 for the last quarter.
`recent_days`	Trading days in the recent move, such as 1 for the last day alone.
`garch_years`	Years of history to fit the GARCH model to.

forecast_days	Trading days to forecast, the weekdays after the last day.
level	Probability of the forecast ranges, and of the value at risk: 0.95 is the 95% range and the worst 5% of days.
volume_days	Trading days before each day whose median volume the day's volume is compared with.

Value

NULL when the history is not longer than a year and the longer of month_days and recent_days, or the price did not move at all in the year before the month or the last days, otherwise a list with

- index, first_date and last_date;
- month: a list of the month's measures, see the code;
- recent: a list of the measures of the last days, with a days data frame of each day's return, GARCH measures, volume, volume_ratio and volume_one_in_days (NA without volume);
- garch: the fitted model, or NULL when it could not be fitted or there are fewer than 500 days before the last days;
- forecast: NULL without a model, otherwise a list with days, a data frame with a row per day ahead: its date, sigma (the daily standard deviation the model expects, in percent) and the closes low, low_50, median, high_50 and high of the ranges (see garch_forecast()); sigma and sigma_long_run, the daily standard deviation expected for the next day and in the long run; half_life, the days for the volatility to get half-way back to the long run; var and es, the value at risk and expected shortfall of the next day; and backtest, the days, exceedances and p_value of the Kupiec test;
- history_risk: var and es of the daily returns of the last days (252) trading days, in percent;
- series: a data frame with a row per day and the columns date, close, volume, return_pct, volume_ratio (see volume_ratio()), volatility_month (of the month_days days up to the day, annualized, in percent), garch_sigma (the daily standard deviation the model expected, in percent), garch_z (the day's move in those standard deviations) and garch_var (the model's value at risk for the day, in percent);
- settings: month_days, recent_days, garch_years, forecast_days, level and volume_days.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
analysis <- volatility_analysis(spx)
analysis$month$level
analysis$recent$days
analysis$forecast$days
volatility_text(analysis)
# the volatility of the last week, and the move of the last day
volatility_text(volatility_analysis(spx, month_days = 5, recent_days = 1))$month$headline
```

`volatility_text` *The volatility analysis in words*

Description

Sentences built from the numbers of `volatility_analysis()`, for the Volatility tab.

Usage

```
volatility_text(analysis)
```

Arguments

`analysis`	The result of `volatility_analysis()`.

Value

A list with `month`, `recent` and `forecast`, each with a `headline` and `points`, a character vector of sentences.

Examples

```
spx <- get_history("S&P 500", as.Date("1950-01-01"))
volatility_text(volatility_analysis(spx))
```

Index

∗ **indices**
    run_app, 15
∗ **stocks**
    run_app, 15
∗ **timeseries**
    run_app, 15

drawdown, 2
drawdown(), *4*, *11*, *14*
drawdown_analysis, 3
drawdown_analysis(), *5*, *11*
drawdown_episodes, 4
drawdown_episodes(), *3*, *4*
drawdown_table, 5
drawdown_table(), *11*
drawdown_text, 5

garch_fit(), *20*
garch_forecast(), *20*, *21*
garch_risk(), *20*
get_history, 6
get_history(), *3*, *7–9*, *14*, *17*, *19*, *20*
get_indices, 7
get_indices(), *6*, *10*

index_summary, 7
index_summary(), *10*

kupiec_test(), *20*

monthly_returns, 8
monthly_returns(), *12*, *19*
moving_average, 9
moving_average(), *14*

normalize_prices, 9
normalize_prices(), *11*

overview_table, 10

plot_compare, 11
plot_drawdowns, 11
plot_drawdowns(), *5*
plot_returns_calendar, 12
plot_seasonality, 13
plot_seasonality(), *17*
plot_timeseries, 13
plot_volatility, 14

run_app, 15

seasonal_returns, 19
seasonal_returns(), *16*, *17*
seasonality_analysis, 16
seasonality_analysis(), *13*, *17*, *18*
seasonality_table, 17
seasonality_text, 18
shiny::shinyApp(), *15*, *16*

volatility_analysis, 20
volatility_analysis(), *14*, *15*, *22*
volatility_text, 22
volume_ratio(), *20*, *21*